

EX NO. 3**ASSET MANAGEMENT USING HYPERLEDGER FABRIC CHAINCODE AIM**

To perform asset management operations such as creating, reading, updating, transferring, and deleting assets using JavaScript chaincode in a Hyperledger Fabric network.

SOFTWARE REQUIREMENTS

- Ubuntu / Kali Linux
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Curl
- jq
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images

HARDWARE REQUIREMENTS

- **Processor:** Intel Core i3 or above
- **RAM:** Minimum 8 GB
- **Storage:** Minimum 20 GB free disk space
- **Internet**

Connection PROCEDURE**Step 1: Navigate to the Hyperledger Fabric Test**

Network Open the terminal and navigate to the Fabric test network. `cd ~/fabric-dev/fabricsamples/test-network`

Step 2: Stop Any Existing Fabric Network

To ensure that no previous network is running, execute:

./network.sh down

Step 3: Start the Fabric Network

Start the Fabric test network with Certificate Authorities and create the application channel.

./network.sh up createChannel -ca

The command performs the following operations:

- Starts the Orderer.
- Starts Org1 and Org2 peers.
- Starts Certificate Authorities.
- Generates cryptographic materials.
- Creates the application channel.

Step 4: Deploy the Asset Transfer Chaincode

Deploy the JavaScript Asset Transfer Basic chaincode.

./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript The command performs:

- Chaincode packaging
- Chaincode installation
- Chaincode approval
- Chaincode commitment

Step 5: Configure Org1 Peer Environment

Load the Org1 peer environment. source

setOrg1.sh

```
vboxuser@ubuntu: ~/fabric-dev/fabric-samples/test-network
vboxuser@Ubuntu: ~/fabric-dev/fabric-samples/test-network$
vboxuser@Ubuntu: ~/fabric-dev/fabric-samples/test-network$ cd ~/fabric-dev/fabric-sam
ples/test-network
./network.sh down
./network.sh up createChannel -ca
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chainc
ode-javascript
source setOrg1.sh
Using docker and docker compose
Stopping network
[+] down 10/10
✓ Container ca_org2 Removed 0.8s
✓ Container orderer.example.com Removed 1.8s
✓ Container ca_org1 Removed 0.8s
✓ Container peer0.org2.example.com Removed 2.8s
✓ Container peer0.org1.example.com Removed 2.4s
✓ Container ca_orderer Removed 0.8s
✓ Network fabric_test Removed 0.3s
✓ Volume compose_orderer.example.com Removed 0.0s
✓ Volume compose_peer0.org2.example.com Removed 0.1s
✓ Volume compose_peer0.org1.example.com Removed 0.1s
```

NAME: VAISHNAVI E

ROLL NO: 231901059

If the helper script is not available, use:

```
export      PATH=${PWD}/../bin:$PATH
export
FABRIC_CFG_PATH=${PWD}/../config/ export
CORE_PEER_TLS_ENABLED=true export
CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/
msp export
CORE_PEER_ADDRESS=localhost:7051
export
ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

Step 6: Set TLS Certificate Paths for Both Organizations

Set the TLS certificate paths for Org1 and

```
Org2. export
ORG1_TLS=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
ORG2_TLS=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
```

These certificates are required because the default channel endorsement policy requires transactions to be endorsed by both organizations.

NAME: VAISHNAVI E

ROLL NO: 231901059

```
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ export ORG1_TLS=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export ORG2_TLS=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$
```

ASSET MANAGEMENT OPERATIONS

Step 7: Initialize the Ledger

Initialize the ledger with the predefined assets.

Run this command as a single line:

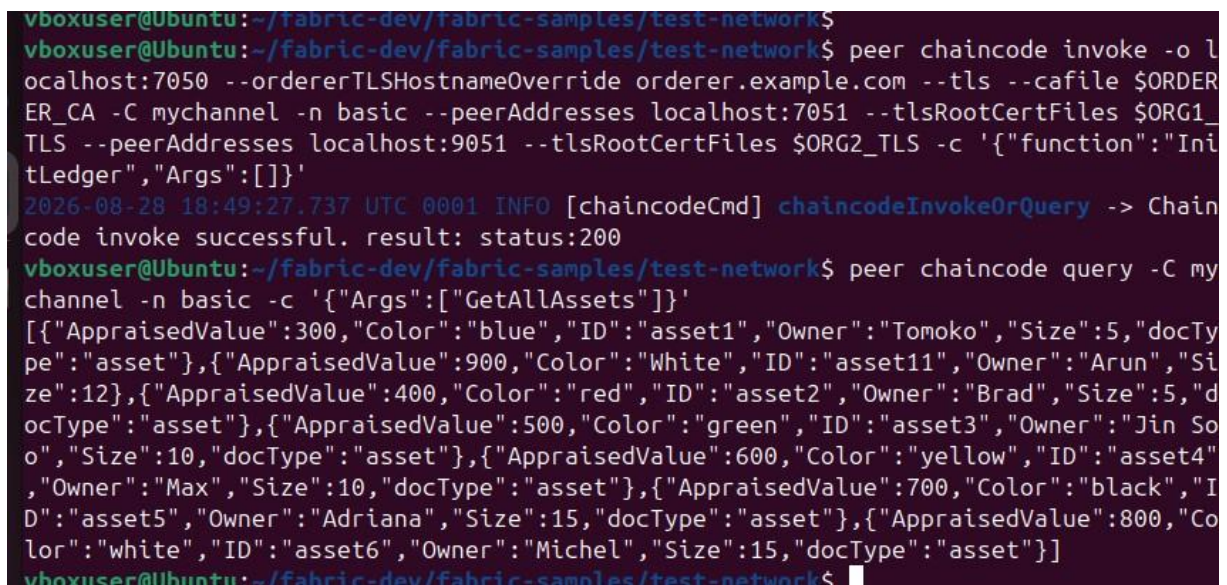
```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -
peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses
localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"function":"InitLedger","Args":[]}'
```

Step 8: Query All Existing Assets

Display all assets currently stored in the ledger. peer

```
chaincode query -C mychannel -n basic -c
```

'{"Args":["GetAllAssets"]}' The command displays the existing asset records.



```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o l
localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDER
ER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_
TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"function":"Ini
tLedger","Args":[]}'
2026-08-28 18:49:27.737 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chain
code invoke successful. result: status:200
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C my
channel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docTy
pe":"asset"}, {"AppraisedValue":900,"Color":"White","ID":"asset11","Owner":"Arun","Si
ze":12}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"d
ocType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin So
o","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4"
,"Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","I
D":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Co
lor":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Step 9: Create a New Asset Create

a new asset named asset11.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -
peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses
localhost:9051 --tlsRootCertFiles $ORG2_TLS -c
'{"function":"CreateAsset","Args":["asset11","White","12","Arun","900"]}'
```

The asset contains the following information:

- **Asset ID:** asset11
- **Color:** White

- **Size:** 12
- **Owner:** Arun
- **Appraised Value:** 900

Step 10: Read the Created Asset

Verify that asset11 was successfully created.

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
```

Expected Output:

```
{
  "ID": "asset11",
  "Color": "White",
  "Size": 12,
  "Owner": "Arun",
  "AppraisedValue": 900
}
```



```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"function":"CreateAsset","Args":["asset11","White","12","Arun","900"]}'
Error: endorsement failure during invoke. response: status:500 message:"The asset as set11 already exists"
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
{"AppraisedValue":900,"Color":"White","ID":"asset11","Owner":"Arun","Size":12}
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$
```

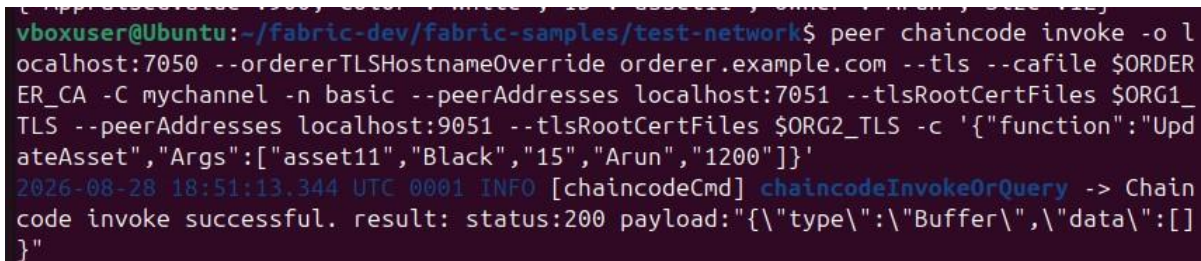
Step 11: Update the Asset

Update the details of asset11. The new values are:

- **Color:** Black
- **Size:** 15
- **Owner:** Arun • **Appraised Value:**

1200 Run:

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -
peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses
localhost:9051 --tlsRootCertFiles $ORG2_TLS -c
'{"function":"UpdateAsset","Args":["asset11","Black","15","Arun","1200"]}'
```



```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o l
ocalhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDER
ER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_
TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"function":"Upd
ateAsset","Args":["asset11","Black","15","Arun","1200"]}'
2026-08-28 18:51:13.344 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chain
code invoke successful. result: status:200 payload:"{\"type\":\"Buffer\",\"data\":[]
```

Step 12: Verify the Updated Asset

Query asset11 again.

```
peer chaincode query -C mychannel -n basic -c
'{"Args":["ReadAsset","asset11"]}'
```

Expected Output:

```
{
  "ID": "asset11",
  "Color": "Black",
  "Size": 15,
  "Owner": "Arun",
  "AppraisedValue": 1200
}
```

Step 13: Transfer Asset Ownership

Transfer ownership of asset11 from Arun to David.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -
peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses
localhost:9051 --tlsRootCertFiles $ORG2_TLS -c
'{"function":"TransferAsset","Args":["asset11","David"]}'
```

Expected Output:

Chaincode invoke successful

Step 14: Verify the New Owner

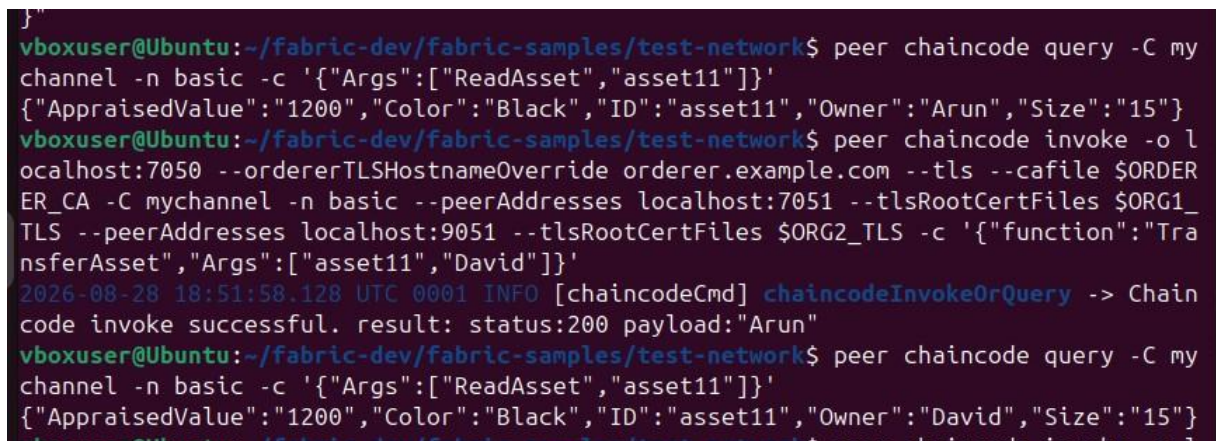
Query asset11 to verify that ownership has changed.

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
```

Expected Output:

```
{
  "ID": "asset11",
  "Color": "Black",
  "Size": 15, "Owner":
  "David",
  "AppraisedValue": 1200
}
```

The ownership of asset11 has been successfully transferred from **Arun** to **David**.



```
}
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C my
channel -n basic -c '{"Args":["ReadAsset","asset11"]}'
{"AppraisedValue": "1200", "Color": "Black", "ID": "asset11", "Owner": "Arun", "Size": "15"}
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o l
ocalhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDER
ER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_
TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"function": "Tra
nsferAsset", "Args": ["asset11", "David"]}'
2026-08-28 18:51:58.128 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chain
code invoke successful. result: status:200 payload:"Arun"
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C my
channel -n basic -c '{"Args":["ReadAsset","asset11"]}'
{"AppraisedValue": "1200", "Color": "Black", "ID": "asset11", "Owner": "David", "Size": "15"}
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o l
```

Step 15: Delete the Asset Delete

asset11 from the ledger.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -
peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses
localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"function": "DeleteAsset", "Args": ["asset11"]}'
```

Expected Output:

Chaincode invoke successful

Step 16: Verify Asset Deletion

Try to read asset11 after deletion.

```
peer chaincode query -C mychannel -n basic -c
 '{"Args":["ReadAsset","asset11"]}'
```

This confirms that the asset was successfully deleted from the ledger.

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOv
erride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCert
Files $ORG1_TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"function":"TransferAsset","Args":["ass
et11","David"]}'
2026-08-28 18:51:58.128 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: stat
us:200 payload:"Arun"
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadA
sset","asset11"]}'
{"AppraisedValue":"1200","Color":"Black","ID":"asset11","Owner":"David","Size":"15"}
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOv
erride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCert
Files $ORG1_TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"function":"DeleteAsset","Args":["asset
11"]}'
2026-08-28 18:52:17.893 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: stat
us:200 payload:{"type":"Buffer","data":[]}
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadA
sset","asset11"]}'
Error: endorsement failure during query. response: status:500 message:"The asset asset11 does not exist"
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$
```

Step 17: Stop the Fabric Network

After completing all asset management operations, stop the Fabric network.

./network.sh down

FLOW OF THE EXPERIMENT

Start Fabric Network

|



Create Application Channel

|



Deploy Chaincode

|



Initialize

Ledger

|



NAME: VAISHNAVI E

ROLL NO: 231901059

Create

Asset

|



Read Asset



Update

Asset

|



Read Updated Asset

|



Transfer Ownership

|



Verify New Owner

|



Delete

Asset

|



Verify Deletion

|



Stop Network

RESULT

NAME: VAISHNAVI E

ROLL NO: 231901059

The asset management operations were successfully performed using Hyperledger Fabric JavaScript chaincode. A new asset named **asset11** was created with the specified details, successfully read from the ledger, updated, and transferred from **Arun** to **David**. Finally, the asset was deleted and its absence was verified. Thus, the complete asset lifecycle operations were successfully implemented and executed on the Hyperledger Fabric network.